EE5203 L02 Coding Practice

Write your own programs with values, decisions, loops, and arrays

Basri Erdogan

Ungraded practice, but you submit it on CATS. Your instructor checks who worked on it. Eleven small programs. You write every one yourself, starting from empty code. You only need what L02 taught: `int`, `uint16_t`, arrays, `for`, `while`, `if` / `else if` / `else`, `&&`, `||`, `/` and `%`. You do not need `printf`.

You do not need the board. Problems P1 to P10 run in your web browser on **OnlineGDB**. Only P11 needs the Nucleo board.

Expected time: about 2 hours. Do a few problems at a time.

How to work on OnlineGDB

OnlineGDB is a free C compiler with a debugger. It runs in the browser. You do not need to install anything or create an account.

1. Open **onlinegdb.com**.
2. Set the **Language** menu at the top of the page to **C**.
3. Delete the example code. Type this template:

```c
#include <stdint.h>

int main(void)
{
    /* your program here */

    return 0;
}
```

   `#include <stdint.h>` is needed for `uint16_t`. `main` is where every C program starts. On the board, the same place is `USER CODE BEGIN 2`.

4. Read the task. Write your program between the braces, above `return 0;`. Write the code yourself. Do not copy code from the lecture.
5. Click to the left of the line number of `return 0;`. A red dot appears. This is your breakpoint.
6. Press **Debug** (not **Run**). In the debug panel at the bottom, press **start**.
7. The program stops at the red dot. Read your variables in the **Local Variables** panel. Click an array to see its elements.
8. Compare them with the table in Check your results at the end.
9. If a value is wrong, set the breakpoint on the first line of your program instead. Press **step over** again and again. Watch the variables change after each line. Find the first variable that goes wrong.

Tips

- Before each new problem, delete the previous problem or put it inside `/* ... */`. Two problems can use the same variable name, such as `sum`.
- A variable shows a strange number before your program stores a value in it. This is normal. It is whatever was in that memory before.

- OnlineGDB does not keep your code after you close the page. Copy each finished program into a text file on your computer.
- When you have your board, you can run the same programs in your L02 project. Put them inside `USER CODE BEGIN 2` in `main.c`, before `while (1)`.

What to submit

Submit **one PDF file** on CATS. Name it `L02_Practice_<student number>.pdf`. Check CATS for the due date.

For each problem from P1 to P10, put these in the PDF:

1. **Your code**, copied as text. Do not use a photo of the screen.
2. **One screenshot** of OnlineGDB, stopped at your breakpoint. The screenshot must show your code and the **Local Variables** panel.
3. **Your answer** to the *Think about it* or *Test it again* part, if the problem has one. One or two sentences are enough.

P11 is not submitted. The challenge in P10 is optional.

It is fine if some of your values are wrong. Submit what you have and write one sentence about what went wrong. You may discuss ideas with classmates, but write your own code. Copied code is easy to see.

Part A - Arithmetic

P1. Split seconds into hours, minutes, and seconds

A timer shows `7385` seconds. Write a program that splits this number into hours, minutes, and seconds. Use only `/` and `%`.

```
int total_s = 7385;
```

Your program must end with these variables: `hours`, `minutes`, `seconds`.

Test it again with `total_s` equal to `59`, `3600`, and `86399`.

P2. Take a number apart, digit by digit

Write a program that looks at the digits of `n` one at a time, from the right. Use a `while` loop. `n % 10` gives the last digit. `n / 10` removes the last digit.

```
int n = 40718;
```

Your program must end with these variables:

- `digit_sum`: all the digits added together
- `digit_count`: how many digits `n` has
- `reversed`: the digits in the opposite order

Think about it: Set `n = 0`. What does your program give for `digit_count`? The number `0` has one digit. Add one `if` after the loop to fix this case.

Part B - Decisions

P3. Battery level from the voltage

A battery voltage is measured in millivolts (mV). Give it a level from 0 to 3:

Voltage	Level
below 3300 mV	0 (empty)
3300 to 3599 mV	1 (low)
3600 to 3999 mV	2 (OK)
4000 mV or more	3 (full)

Six test voltages are stored in an array. Use one `for` loop and one `if` / `else if` / `else` chain. Store each answer in `level`.

```
int mv[6] = {3299, 3300, 3599, 3600, 3999, 4000};
int level[6];
```

Your program must end with the array `level` filled.

These test values sit on the boundaries on purpose. Check every one.

P4. Fan on or off

A fan turns on in two cases:

- the temperature is above 30 °C, **or**
- the temperature is above 25 °C **and** the humidity is above 70 %.

Write a program that sets `fan[i]` to `1` (on) or `0` (off) for each measurement. Also count how many times the fan is on.

```
int temp[6] = {32, 28, 28, 24, 26, 31};
int hum[6]  = {40, 75, 60, 90, 71, 20};
int fan[6];
```

Your program must end with these variables: the array `fan`, and `fan_on_count`.

Part C - Loops

P5. How many numbers until the sum is over 1000?

Add `1 + 2 + 3 + ...` until the sum is **greater than** `1000`. Use a `while` loop. You do not know the number of repetitions before you start. This is why `while` fits better than `for` here.

Your program must end with these variables:

- `n`: the last number you added
- `sum`: the sum at that moment

Test it again with `5000` in place of `1000`.

P6. Powers of two in `uint16_t`

Fill an array with the powers of two: `1, 2, 4, 8, ....` Use a `for` loop. Each element is two times the element before it.

```
uint16_t power[16];
```

Your program must end with the array `power` filled, from `power[0]` to `power[15]`.

Think about it: after the loop, add this line:

```
uint16_t next = power[15] * 2;
```

Predict the value of `next` before you run. Then check it in the debugger. Explain the result in one sentence. Hint: what is the largest value a `uint16_t` can hold?

Part D - Arrays

P7. Fastest, slowest, and where they are

Ten speed readings are stored in an array. Write one loop that finds:

- the largest value and its index
- the smallest value and its index
- the sum of all values

The largest value appears two times. Keep the **first** index.

Then compute the average **after** the loop. Write a second loop that counts the readings above the average.

```
int speed[10] = {42, 57, 39, 61, 58, 61, 35, 44, 50, 47};
```

Your program must end with these variables: `max_value`, `max_index`, `min_value`, `min_index`, `sum`, `average`, `above_average`.

Think about it: what should `max_value` start at? Try `0`, then try `speed[0]`. Both work here. Which one still works if all speeds are negative?

P8. Average of the good readings only

A distance sensor sends 10 readings in millimetres. Some readings are bad:

- `0` means the sensor did not get an echo.
- Anything above `1500` is out of range.

Only readings from `1` to `1500` are good. Compute the average of the good readings only.

```
uint16_t dist_mm[10] = {412, 0, 398, 405, 0, 0, 2000, 401, 0, 398};
```

Your program must end with these variables:

- `good_count` and `bad_count`
- `good_sum`: the sum of the good readings
- `average`: `good_sum / good_count`
- `average_rounded`: the average rounded to the nearest whole number

Integer division always rounds down. To round to the nearest number, use `(good_sum + good_count / 2) / good_count`.

Think about it: what happens if **all** readings are bad? Add an `if` so your program never divides by zero.

P9. Reverse an array in place

Reverse the order of the elements **inside the same array**. Swap the first and the last element, then the second and the second-to-last, and so on. A swap needs a third variable to hold one value for a moment.

```
int a[7] = {1, 2, 3, 4, 5, 6, 7};
```

Your program must end with `a` reversed.

Think about it: how many swaps do you need for 7 elements? Run your loop for all 7 indexes instead. What does the array look like at the end, and why?

P10. Is the array in order?

Check if every element is greater than or equal to the element before it.

```
int b[8] = {5, 9, 12, 12, 20, 18, 25, 30};
```

Your program must end with these variables:

- `in_order`: `1` if the array is in order, `0` if it is not
- `first_problem`: the first index `i` where `b[i]` is smaller than `b[i - 1]`, or `-1` if there is no problem

Start the loop at `i = 1`. Only record the **first** problem. The `&&` operator helps here.

Challenge (optional): put the array in order. Walk through the array and swap two neighbours when they are in the wrong order. Repeat the walk until no swap happens. You will need one loop inside another loop.

Part E - On the board

P11. A seconds counter in the forever loop

Do this problem when you have your Nucleo board. It uses `HAL_Delay` and the LED. These do not exist on OnlineGDB.

This problem uses `while (1)`. Put your code **inside** the generated `while (1)` loop, in `USER CODE BEGIN 3`.

Every pass through the loop:

1. waits one second with `HAL_Delay(1000);`
2. adds one to `elapsed_s`
3. computes `minutes` and `seconds` from `elapsed_s`
4. toggles the green LED **only** when `elapsed_s` is a multiple of 5, and adds one to `toggle_count` when it does

Use the toggle line from Lab 1: `HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);`

Declare `elapsed_s` and `toggle_count` in `USER CODE BEGIN 2`, with the value `0`. Do not declare them inside the loop. Run the program without breakpoints. The LED should change every 5 seconds. After a little more than one minute, press **Pause** and read the variables.

Think about it: what happens to `elapsed_s` if you declare it inside `while (1)`? Try it and look at the debugger.

Check your results

Check only after you run your own program.

Problem	Correct final values
P1	`hours = 2`, `minutes = 3`, `seconds = 5`. Tests: `59` → 0, 0, 59 · `3600` → 1, 0, 0 · `86399` → 23, 59, 59
P2	`digit_sum = 20`, `digit_count = 5`, `reversed = 81704`, and `n = 0` after the loop. With `n = 0`, the loop never runs, so `digit_count` is `0` until you add the fix
P3	`level = {0, 1, 1, 2, 2, 3}`
P4	`fan = {1, 1, 0, 0, 1, 1}`, `fan_on_count = 4`
P5	`n = 45`, `sum = 1035` (at `n = 44` the sum is only 990). With `5000`: `n = 100`, `sum = 5050`
P6	`power[10] = 1024`, `power[15] = 32768`, `next = 0`. `65536` does not fit in `uint16_t`; the largest value is `65535`
P7	`max_value = 61`, `max_index = 3`, `min_value = 35`, `min_index = 6`, `sum = 494`, `average = 49`, `above_average = 5`. If you get `max_index = 5`, you used `>=` instead of `>`
P8	`good_count = 5`, `bad_count = 5`, `good_sum = 2014`, `average = 402`, `average_rounded = 403`
P9	`a = {7, 6, 5, 4, 3, 2, 1}` with 3 swaps. With 7 swaps, every pair is swapped two times, so the array is back to `{1, 2, 3, 4, 5, 6, 7}`
P10	`in_order = 0`, `first_problem = 5` (`18` comes after `20`). Challenge: `{5, 9, 12, 12, 18, 20, 25, 30}`
P11	Your numbers depend on when you press Pause. For example, at `elapsed_s = 75`: `minutes = 1`, `seconds = 15`, `toggle_count = 15`. Inside `while (1)`, `elapsed_s` starts from `0` again on every pass, so it never grows past `1`